

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

Name:K.mrudhula

Reg number:192372290

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with `Content-Length` and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
<head>
<title>TestPage</title>
</head>
<body>
```

```
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser

2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

*I **mrudhula.k(192372290)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: mrudhula.k

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions

Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand
---------	-----	--	-----------------------------------	-----------------------------------	---

1: HTTP Response Analysis

A web server returns the following HTTP response header:
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

Questions:

- 1. Identify any incorrect or invalid fields in the header.
- 2. Analyze the issue with Content-Length and explain its impact.
- 3. Determine whether the status code matches the response correctness.
- 4. Suggest corrections for the identified errors.

Solution:

1. HTTP Response Analysis

1. Identify Any Incorrect or Invalid Fields in the Header

Given HTTP Response:

HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

Issues:

- Content-Length: -150 is invalid because Content-Length cannot be negative.
- All other fields (HTTP/1.1, 200 OK, Date, Content-Type, and Connection) are valid.

2. Analyze the Issue with Content-Length and Explain Its Impact

Problem:

- **Content-Length** specifies the size of the response body in **bytes**.
- A negative value (**-150**) is not allowed in HTTP.

Impact:

- Browsers may fail to load the webpage correctly.
- The client may not know where the response ends.
- Some browsers may reject the response or display an error.
- Data transfer may be interrupted or incomplete.

3. Determine Whether the Status Code Matches the Response Correctness

- **Status Code:** **200 OK**
- **Meaning:** The request was processed successfully.

However, because the **Content-Length** header is invalid, the response is **not completely**

correct. Conclusion:

- The **200 OK** status indicates success.
- But the invalid **Content-Length** makes the response malformed.
- The server should send a valid **Content-Length** before returning **200 OK**.

4. Suggest Corrections for the Identified Errors

Corrected HTTP Response

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

Improvements

- Set **Content-Length** to the actual number of bytes in the response body.
- Verify header values before sending the response.
- Follow HTTP/1.1 standards for valid header formatting.
- Test the server response using browser developer tools or HTTP testing tools.

Result

The HTTP response contains an invalid **Content-Length** value. Replacing it with the correct positive byte count makes the response standards-compliant and ensures browsers can process the response correctly.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid

Solution:

2. DOM Structure Analysis

1. Identify Missing or Improperly Closed Elements

The given HTML has the following errors:

- The `<p>` tag is not closed.
- The `<div>` tag closes before the `<p>` tag is closed.
- Proper nesting of elements is missing.

Given Code:

```
<html>
<head>
<title>TestPage</title>
</head>

<body>
<div>
<p>HelloWorld
</div>
</body>
</html>
```

2. Analyze How the Browser Will Interpret This Structure

Modern browsers automatically correct invalid HTML.

- The browser will automatically insert the missing `</p>` tag before `</div>`.
- It creates a valid DOM tree even though the source HTML is incorrect.
- Different browsers usually display the same output, but relying on automatic correction is not recommended.

The browser interprets it as:

HTML:

```
<div>
  <p>HelloWorld</p>
</div>
```

3. Determine the Impact on Rendering and Layout

- The page will usually display **"HelloWorld"** correctly.
- Invalid HTML may lead to inconsistent rendering across browsers.
- CSS styles may not behave as expected.
- JavaScript DOM manipulation may produce unexpected results.
- Poor HTML structure reduces maintainability and accessibility.

4. Suggest Corrections to Make the DOM Valid

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TestPage</title>
</head>

<body>

  <div>
    <p>HelloWorld</p>
  </div>

</body>
</html>
```

Result

The original HTML contains an unclosed `<p>` tag. Browsers automatically correct it during parsing, but using properly nested and closed tags creates a valid DOM, improves browser compatibility, and ensures

consistent rendering and easier maintenance.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

- 1. Interpret the differences in data transmission between GET and POST.
- 2. Analyze which method is more suitable for login and why.
- 3. Identify security risks in using GET for authentication.
- 4. Suggest best practices for secure login implementation.

Solution:

3. GET vs POST in Login System

1. Interpret the Differences in Data Transmission Between GET and POST

Feature		GET	POST
Data	Transmission	Sent in the URL as query parameters	Sent in the HTTP request body
	Data Visibility	Visible in the browser URL	Hidden from the URL
Data Size	Limited	Can send larger amounts of data	Security
	Less secure	More secure	than GET
Common Use		Retrieving data	Submitting sensitive data (login, registration)

2. Analyze Which Method is More Suitable for Login and Why

The **POST** method is more suitable for login because:

- It sends the username and password in the **request body**, not in the URL.
- Login credentials are not visible in the browser's address bar.

- It provides better security when used with **HTTPS**.
- It prevents sensitive data from being stored in browser history or bookmarks. **3.**

Identify Security Risks in Using GET for Authentication

Using **GET** for login has the following risks:

- Username and password are visible in the URL.
- Credentials may be stored in browser history.
- URLs can be bookmarked with sensitive information.
- Credentials may appear in server logs.
- URLs can be easily shared or exposed, increasing the risk of unauthorized access.

Example:

/login?user=admin&pass=1234

4. Suggest Best Practices for Secure Login Implementation

- Use the **POST** method for login forms.
- Use **HTTPS** to encrypt data during transmission.
- Validate user input on both the client and server.
- Store passwords securely using hashing algorithms.
- Use secure session management after successful login.
- Implement strong password policies.
- Protect against SQL Injection and Cross-Site Scripting (XSS).
- Enable Multi-Factor Authentication (MFA) when possible.

Result

The **POST** method is the preferred choice for login systems because it sends credentials in the request body instead of the URL. When combined with **HTTPS**, proper input validation, password hashing, and secure session management, it provides a much safer authentication process than the **GET** method.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the `<img>` tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

Solution:

4. HTML Code Inspection

1. Identify Missing Closing Tags

The given HTML has the following missing closing tags:

- `</h1>` is missing.
- `</p>` is missing.
- The `<img>` tag should include an `alt` attribute for accessibility.

2. Analyze Issues with the `<img>` Tag

The `<img>` tag has the following issue:

- It does not contain an `alt` attribute.
- The `alt` attribute provides alternative text if the image cannot be displayed and improves accessibility for screen readers.

Current tag:

```

```

Improved tag:

```

```

3. Determine How Browsers Will Handle These Errors

- Browsers automatically insert missing closing tags during HTML parsing.
- The page will usually display correctly despite the errors.
- Missing closing tags can cause unexpected layout or styling issues.
- The missing `alt` attribute does not stop the image from loading but reduces accessibility and may cause HTML validation warnings.

4. Corrected Version of the Code

```
<!DOCTYPE html>  
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Sample</title>
</head>

<body>

<h1>Welcome</h1>
<p>This is a paragraph.</p>



</body>
</html>
```

Result

The HTML code is corrected by adding the missing `</h1>` and `</p>` closing tags, including an `alt` attribute for the image, and adding standard HTML5 metadata. The corrected code is valid, accessible, and renders consistently across modern web browsers.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Solution:

5. Browser–Server Communication Flow

1. Interpret Each Step in the Communication Flow

Step 1: User enters a URL in the browser

- The user types a website address (URL) such as <https://www.example.com>.

Step 2: DNS lookup occurs

- The browser contacts the **Domain Name System (DNS)** to convert the domain name into the server's IP address.

Step 3: HTTP request is sent

- The browser sends an HTTP/HTTPS request to the web server asking for the required web page.

Step 4: Server processes the request

- The server receives the request, processes it, retrieves data if necessary, and prepares the response.

Step 5: Response is returned

- The server sends an HTTP response containing the status code, headers, and requested content (HTML, CSS, JavaScript, images, etc.).

Step 6: Browser renders the page

- The browser interprets the HTML, downloads required resources, applies CSS, executes JavaScript, and displays the web page to the user.

2. Identify Where Delays Can Occur

Delays may occur at the following stages:

- **DNS lookup** if DNS resolution is slow.
- **Network transmission** due to slow internet or high latency.
- **Server processing** if the server is overloaded or database queries are slow.
- **Response download** when files are large.
- **Browser rendering** if the page contains large images, complex CSS, or heavy JavaScript.

3. Analyze the Role of DNS in the Process

The **Domain Name System (DNS)** translates a domain name (e.g., www.example.com) into an IP address that identifies the web server.

Without DNS:

- The browser would not know where the website is located.
- Communication with the server cannot begin.

Thus, DNS acts as the **bridge between the website name and the server's IP address**.

4. Suggest Optimizations to Improve Performance

- Use **DNS caching** to reduce lookup time.
- Enable **browser caching** for frequently accessed resources.
- Compress files using **Gzip** or **Brotli**.
- Optimize images and other media files.
- Minify HTML, CSS, and JavaScript files.
- Use a **Content Delivery Network (CDN)** to deliver content faster.
- Use **HTTPS** with HTTP/2 or HTTP/3 for improved performance.
- Optimize server and database performance.

Result

The browser-server communication flow starts with entering a URL, followed by DNS resolution, HTTP request, server processing, response delivery, and browser rendering. Performance can be improved through DNS caching, optimized resources, efficient server processing, and the use of CDNs and modern web protocols.